



Java app store
Sun plans to implement the biggest application store on the internet around Java



Yahoo! quits
Yahoo!'s own Java store shuts shop, like its other mysteriously disappearing projects



MS at JavaOne
Microsoft is giving a keynote address at JavaOne, a cheeky move considering the infamous 1990 fallout

The BlueJ Way

Somewhere between *I want to learn Java* and *I know Java*, there is the uncertain period of *Object-oriented Programming makes no sense*. And through those tough times, BlueJ can help you visualise your objects

Nimish Chandiramani
readersletters@thinkdigit.com

Most programmers will tell you that once you've understood the basics of object-oriented programming (OOP), you're set—you'll be ready to tackle any programming language, and master it with ease. And if you're learning Java, BlueJ makes that task much easier. It lets you see the structure of your classes and objects, just like when you're planning your code. It's all Java, but the concepts you'll learn with BlueJ carry forth to all object-oriented languages (and we aren't going to do any advanced Java programming in this article, so the code should be easy to follow).

And now, on to it.

Objects and classes

In the world of OOP, everything is an object. Simply put, the code equivalent of a real-world object. And like real-world objects, these programming objects have properties (information about them), and behaviours (stuff they do). If you're using a web application, for example, it sees you as an object called `yourUserName`, which has a bunch of properties that represent you—a name, an email address, and

GET BLUEJ

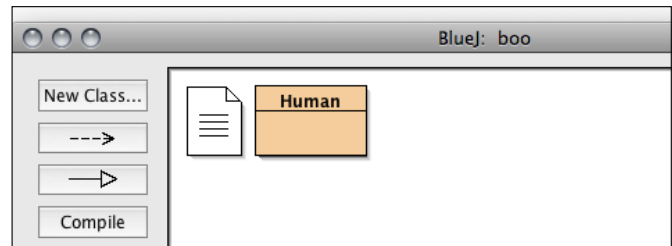
Download BlueJ at www.bluej.org. You'll need Java installed to run it—get that from <http://www.java.com/en/download/index.jsp>. If you're running on Linux or Mac OS X, you've likely already got Java installed.

so on—and another bunch of behaviours—log out, purchase, and other such activities.

A class is the blueprint on which the object is based. In the above example, `yourUserName` is an object, but it's based on a class called `User`, which says that all user objects must have a name and email address, and should have the default behaviours of purchasing stuff and logging out. So if someone else logs on to the site, they are represented by an object called `someoneElseUserName`, which has the same properties and behaviour as `yourUserName`, just different information.

Doing it in BlueJ

Fire up BlueJ, and choose `File > New Project`. Select a directory to save it in, and you'll now see a blank window that's waiting for you to do something. Right-click anywhere in this window and choose `New Class`. Enter a name for this class—`Human`, for example (for we can play



It's a boy!

on `Human` and select `new Human()`. You'll be asked to name your new human, and once you click OK, you'll see this new human appear at the bottom of the window.

When you're done admiring your handiwork, right-click on the `Human` and choose `Remove`.

Properties and behaviour

Now that you know how to create classes, you can start adding properties (called fields in Java) and behaviour to them. Double-click on the `Human` class to see its code. First, let's begin with adding a field for `Humans`. Towards the beginning of the code,

The "public" means that `Age` can be accessed (and thus modified) by any other function, even if it isn't in the `Human` class. That done, you can now focus on the `Human`'s methods.

Every class has a constructor—a function that runs when an object of that class is created—which shares its name with the class. BlueJ's default constructor just sets the value of `x` to 0, so let's make that less boring:

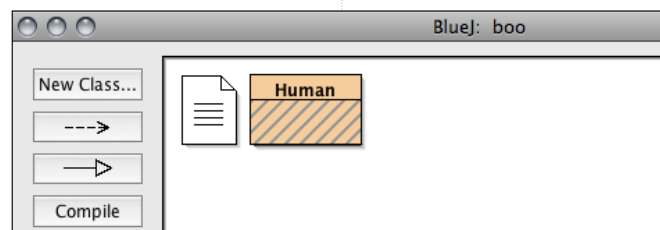
```
public Human ()
{
    //just born
    Age = 0;

    // cry baby cry
    System.out.println
("WAAAAAAAAAH!");
}
```

Now, every time you create a new `Human`, it'll cry into a terminal window. Let's also replace the default `sampleMethod` with something more appropriate:

```
public void
saySomething(String
something)
{
    // first words
    System.out.
println(something);
}
```

And now, you're ready to test these behaviours in the wild. Compile your project, and create a new `Human` as before.



You've now got a basic blueprint for your `Human`

God here)—and hit `[Enter]`. You'll now see a little orange box called `Human`, which represents the class.

Before you create your first `Human` object, hit the `Compile` button to compile the code, which turns it into an application. Now right-click

you'll see "private int x;". This declares an integer `x`, but because it's "private", it can only be used by functions within the class. Let's add an age to the `Human` class:

```
public int Age;
```